

OBJECTIVE

To analyze sales performance, revenue drivers, and return behavior in order to understand business patterns, customer dynamics, and potential risks.

Import Required Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_style("whitegrid")
```

Loading the Dataset

```
df_raw = pd.read_csv("/content/online_retail_II.csv")
df = df_raw.copy()
```

Dataset Overview

```
df.shape
```

```
(1067371, 8)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1067371 entries, 0 to 1067370
Data columns (total 8 columns):
 #   Column          Non-Null Count  Dtype  
---  --
 0   Invoice          1067371 non-null object  
 1   StockCode       1067371 non-null object  
 2   Description     1062989 non-null object  
 3   Quantity        1067371 non-null int64   
 4   InvoiceDate     1067371 non-null object  
 5   Price           1067371 non-null float64  
 6   Customer ID    824364 non-null float64   
 7   Country         1067371 non-null object  
dtypes: float64(2), int64(1), object(5)
memory usage: 65.1+ MB
```

```
df.describe()
```

	Quantity	Price	Customer ID	
count	1.067371e+06	1.067371e+06	824364.000000	
mean	9.938898e+00	4.649388e+00	15324.638504	
std	1.727058e+02	1.235531e+02	1697.464450	
min	-8.099500e+04	-5.359436e+04	12346.000000	
25%	1.000000e+00	1.250000e+00	13975.000000	
50%	3.000000e+00	2.100000e+00	15255.000000	
75%	1.000000e+01	4.150000e+00	16797.000000	
max	8.099500e+04	3.897000e+04	18287.000000	

```
df.isnull().sum()
```

```

      0
Invoice      0
StockCode    0
Description  4382
Quantity     0
InvoiceDate  0
Price        0
Customer ID  243007
Country      0

```

dtype: int64

```
df['Quantity'].describe()
```

```

      Quantity
count  1.067371e+06
mean   9.938898e+00
std    1.727058e+02
min    -8.099500e+04
25%    1.000000e+00
50%    3.000000e+00
75%    1.000000e+01
max     8.099500e+04

```

dtype: float64

```
df['Price'].describe()
```

```

      Price
count  1.067371e+06
mean   4.649388e+00
std    1.235531e+02
min    -5.359436e+04
25%    1.250000e+00
50%    2.100000e+00
75%    4.150000e+00
max    3.897000e+04

```

dtype: float64

```

# Negative quantities (returns)
df[df['Quantity'] < 0].head()

```

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country	
178	C489449	22087	PAPER BUNTING WHITE LACE	-12	2009-12-01 10:33:00	2.95	16321.0	Australia	
179	C489449	85206A	CREAM FELT EASTER EGG BASKET	-6	2009-12-01 10:33:00	1.65	16321.0	Australia	
180	C489449	21895	POTTING SHED SOW 'N' GROW SET	-4	2009-12-01 10:33:00	4.25	16321.0	Australia	
181	C489449	21896	POTTING SHED TWINE	-6	2009-12-01 10:33:00	2.10	16321.0	Australia	
182	C489449	22083	PAPER CHAIN KIT RETRO SPOT	-12	2009-12-01 10:33:00	2.95	16321.0	Australia	

```

# Zero or negative prices
df[df['Price'] <= 0].head()

```

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country	
263	489464	21733	85123a mixed	-96	2009-12-01 10:52:00	0.0	NaN	United Kingdom	
283	489463	71477	short	-240	2009-12-01 10:52:00	0.0	NaN	United Kingdom	
284	489467	85123A	21733 mixed	-192	2009-12-01 10:53:00	0.0	NaN	United Kingdom	
470	489521	21646	NaN	-50	2009-12-01 11:44:00	0.0	NaN	United Kingdom	
3114	489655	20683	NaN	-44	2009-12-01 17:26:00	0.0	NaN	United Kingdom	

```
# Missing customers
df[df['Customer ID'].isnull()].shape
```

```
(243007, 8)
```

```
# Weird descriptions
df['Description'].value_counts().head(20)
```

	count
Description	
WHITE HANGING HEART T-LIGHT HOLDER	5918
REGENCY CAKESTAND 3 TIER	4412
JUMBO BAG RED RETROSPOT	3469
ASSORTED COLOUR BIRD ORNAMENT	2958
PARTY BUNTING	2765
STRAWBERRY CERAMIC TRINKET BOX	2613
LUNCH BAG BLACK SKULL.	2529
JUMBO STORAGE BAG SUKI	2434
HEART OF WICKER SMALL	2319
JUMBO SHOPPER VINTAGE RED PAISLEY	2297
60 TEATIME FAIRY CAKE CASES	2271
PAPER CHAIN KIT 50'S CHRISTMAS	2215
LUNCH BAG SPACEBOY DESIGN	2206
REX CASH+CARRY JUMBO SHOPPER	2199
HOME BUILDING BLOCK WORD	2192
WOODEN FRAME ANTIQUE WHITE	2189
LUNCH BAG CARS BLUE	2184
NATURAL SLATE HEART CHALKBOARD	2151
BAKING SET 9 PIECE RETROSPOT	2150
POSTAGE	2115

```
dtype: int64
```

Observations

- Dataset contains 106737 rows and 8 columns
- Missing values present in: description and customer id
- Customer ID has 24300 missing values
- Quantity contains negative values indicating returns
- InvoiceDate is currently in object format and needs conversion

▼ Data Cleaning

```
# Convert date
df['InvoiceDate'] = pd.to_datetime(df['InvoiceDate'])

# Create Total Price
df['TotalPrice'] = df['Quantity'] * df['Price']

# Create separate datasets
```

```
sales_df = df[df['Quantity'] > 0].copy()
returns_df = df[df['Quantity'] < 0].copy()
```

```
# calculating total revenue
total_revenue = (sales_df['Quantity'] * sales_df['Price']).sum()
total_revenue
```

```
np.float64(20814291.998000003)
```

```
# total number of orders in the dataset
total_orders = sales_df['Invoice'].nunique()
total_orders
```

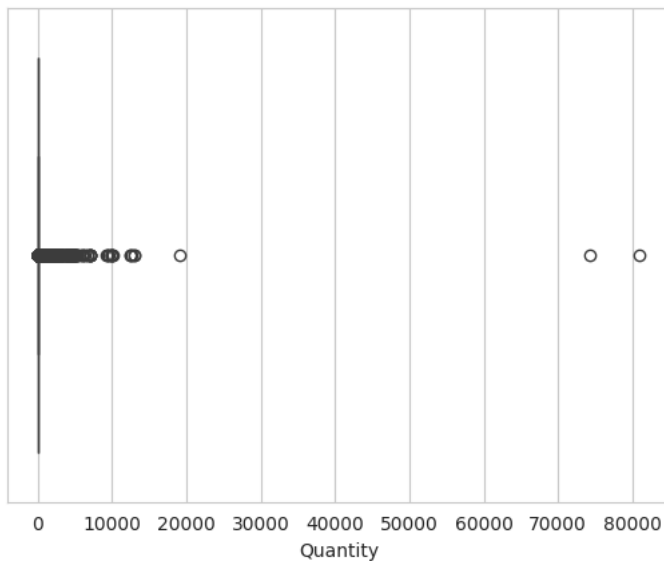
```
41944
```

```
# average value per order
avg_order_value = (sales_df['Quantity'] * sales_df['Price']).groupby(sales_df['Invoice']).sum().mean()
avg_order_value
```

```
np.float64(496.24003428380695)
```

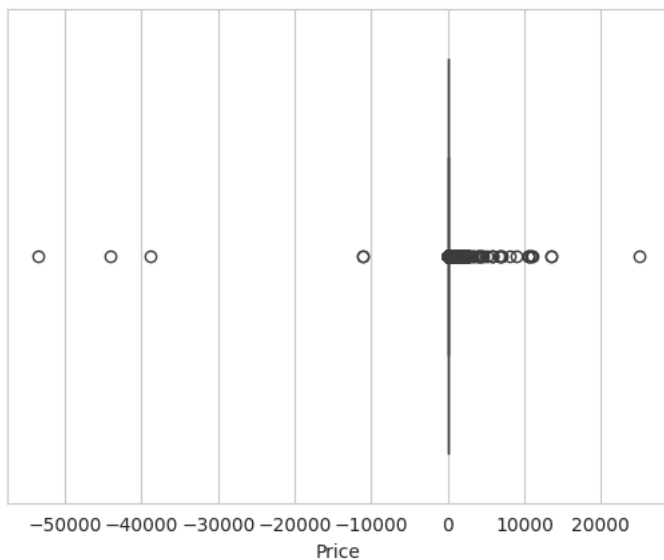
```
sns.boxplot(x=sales_df['Quantity'])
```

<Axes: xlabel='Quantity'>



```
sns.boxplot(x=sales_df['Price'])
```

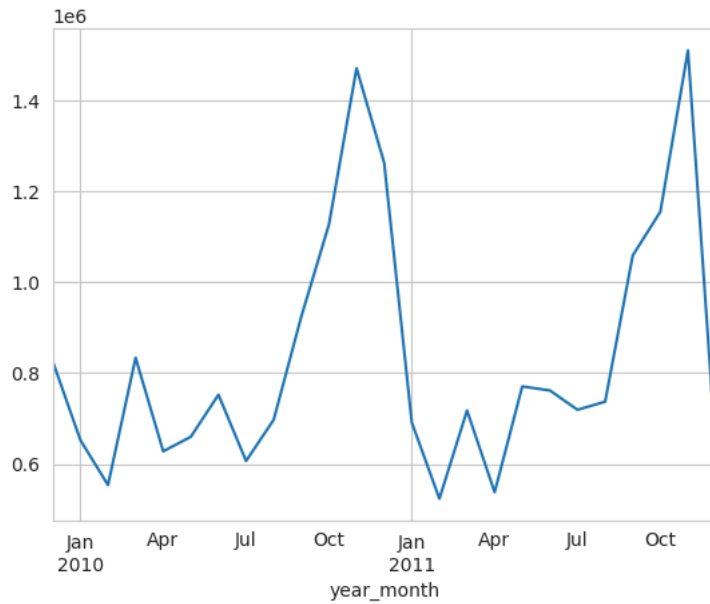
<Axes: xlabel='Price'>



```
sales_df['InvoiceDate'] = pd.to_datetime(sales_df['InvoiceDate'])
sales_df['year_month'] = sales_df['InvoiceDate'].dt.to_period('M')
sales_df['TotalPrice'] = sales_df['Quantity'] * sales_df['Price']
monthly_revenue = sales_df.groupby('year_month')['TotalPrice'].sum()
```

```
monthly_revenue.plot()
```

```
<Axes: xlabel='year_month'>
```



```
# Growth rate
```

```
monthly_revenue.pct_change()
```

TotalPrice

year_month

2009-12	NaN
2010-01	-0.209495
2010-02	-0.151668
2010-03	0.505418
2010-04	-0.246692
2010-05	0.050840
2010-06	0.140047
2010-07	-0.193533
2010-08	0.149327
2010-09	0.325636
2010-10	0.218779
2010-11	0.305101
2010-12	-0.141248
2011-01	-0.452427
2011-02	-0.242611
2011-03	0.370504
2011-04	-0.250587
2011-05	0.432733
2011-06	-0.011416
2011-07	-0.055818
2011-08	0.024739
2011-09	0.436323
2011-10	0.091054
2011-11	0.306947
2011-12	-0.576805

```
dtype: float64
```

Start coding or [generate](#) with AI.

SALES ANALYSIS

```
sales_df['revenue'] = sales_df['Quantity'] * sales_df['Price']
```

Order Behavior

```
total_orders = sales_df['Invoice'].nunique()
```

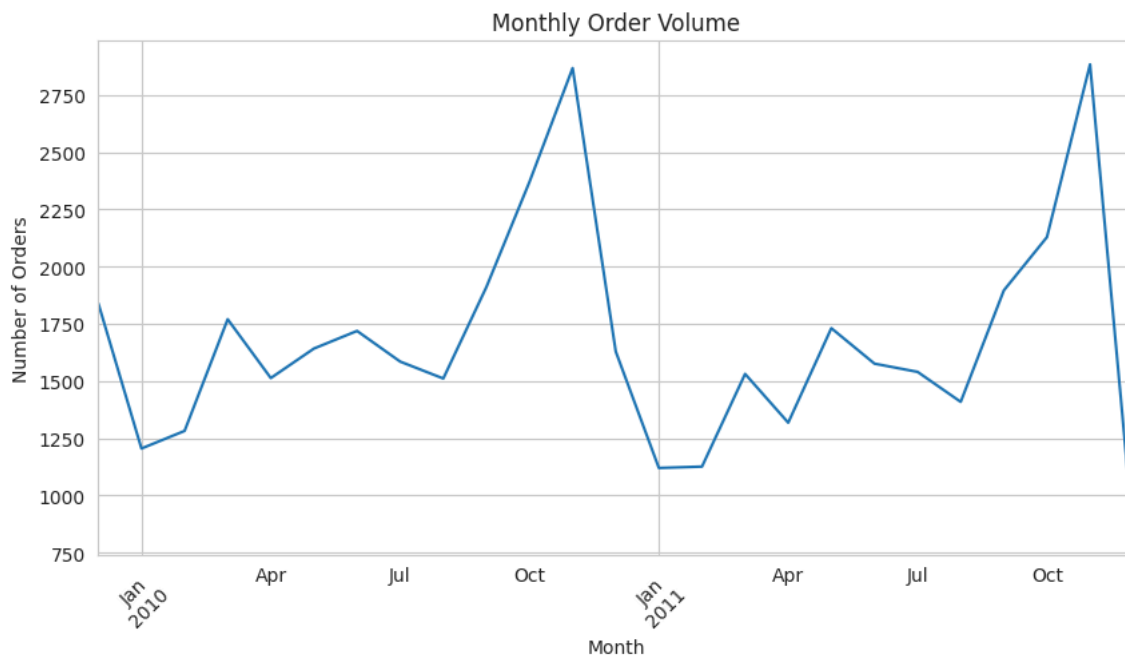
```
monthly_orders = sales_df.groupby('year_month')['Invoice'].nunique()
```

monthly_orders

Invoice	
year_month	
2009-12	1839
2010-01	1205
2010-02	1282
2010-03	1770
2010-04	1513
2010-05	1642
2010-06	1719
2010-07	1585
2010-08	1511
2010-09	1911
2010-10	2371
2010-11	2868
2010-12	1629
2011-01	1120
2011-02	1126
2011-03	1531
2011-04	1318
2011-05	1731
2011-06	1576
2011-07	1540
2011-08	1409
2011-09	1896
2011-10	2129
2011-11	2884
2011-12	839

dtype: int64

```
monthly_orders.plot(figsize=(10,5))
plt.title("Monthly Order Volume")
plt.xlabel("Month")
plt.ylabel("Number of Orders")
plt.xticks(rotation=45)
plt.show()
```



Order volume shows strong seasonality, peaking in November across years, followed by sharp declines in January. The unusually low values in December 2011 suggest incomplete data rather than actual business decline.

Customer Behavior

```
customers = sales_df['Customer ID'].nunique()

orders_per_customer = sales_df.groupby('Customer ID')['Invoice'].nunique()
```

```
orders_per_customer.describe()
```

	Invoice
count	5881.000000
mean	6.287196
std	13.012879
min	1.000000
25%	1.000000
50%	3.000000
75%	7.000000
max	398.000000

dtype: float64

Product Demand

```
product_quantity = sales_df.groupby('Description')['Quantity'].sum().sort_values(ascending=False)
```

```
product_quantity.head(10)
```

	Quantity
Description	
WORLD WAR 2 GLIDERS ASSTD DESIGNS	110249
WHITE HANGING HEART T-LIGHT HOLDER	96688
ASSORTED COLOUR BIRD ORNAMENT	81817
PAPER CRAFT , LITTLE BIRDIE	80995
JUMBO BAG RED RETROSPOT	79285
MEDIUM CERAMIC TOP STORAGE JAR	78033
BROCADE RING PURSE	71440
PACK OF 60 PINK PAISLEY CAKE CASES	56794
60 TEATIME FAIRY CAKE CASES	54716
SMALL POPCORN HOLDER	49984

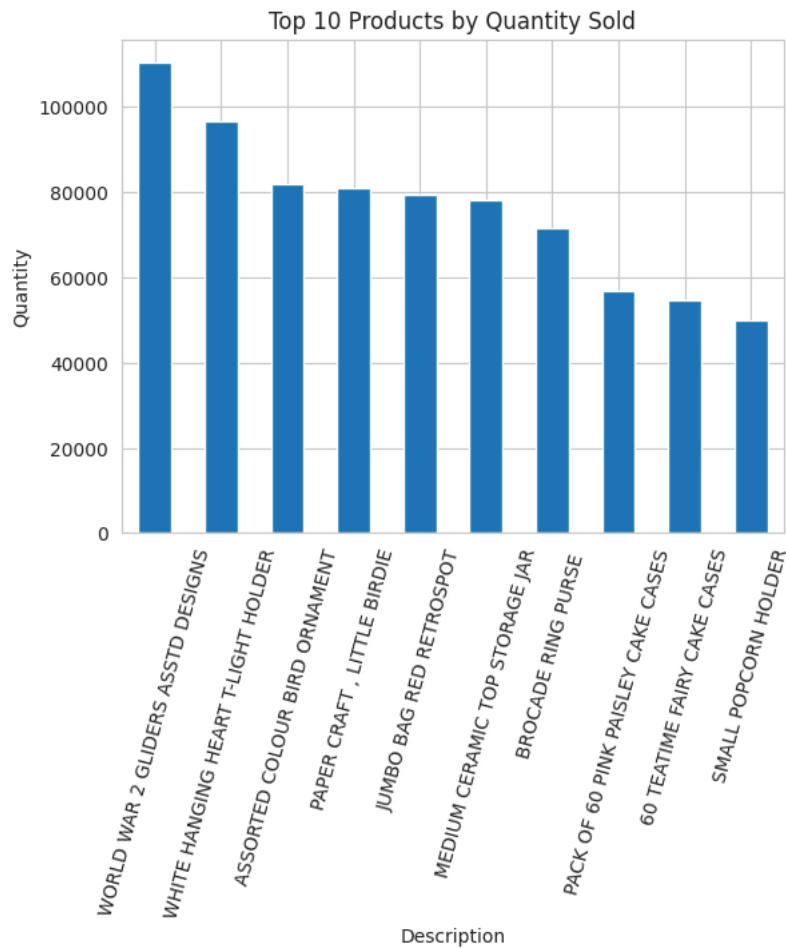
dtype: int64

```
product_quantity.tail(10)
```

	Quantity
Description	
OWLS CHARLOTTE BAG	1
S/2 HYACINTH BULB CANDLES	1
S/3 COLOURS FLOCK TEA TOWELS	1
SPOTTY CAT DOORSTOP	1
SPOTTY COCKEREL DOORSTOP	1
CIRCUS PARADE LUNCHBOX	1
RUSSIAN FOLKART STACKING TINS	1
GOLD CHRISTMAS STOCKING DECORATION	1
I LOVE LONDON MINI RUCKSACK	1
S/6 CHRISTMAS TREE PLACE SETTING	1

dtype: int64

```
product_quantity.head(10).plot(kind='bar')
plt.title("Top 10 Products by Quantity Sold")
plt.ylabel("Quantity")
plt.xticks(rotation=75)
plt.show()
```

Product demand is concentrated among a limited set of items, indicating strong product-level skewness in sales volume.

✓ Basket size (Order Composition)

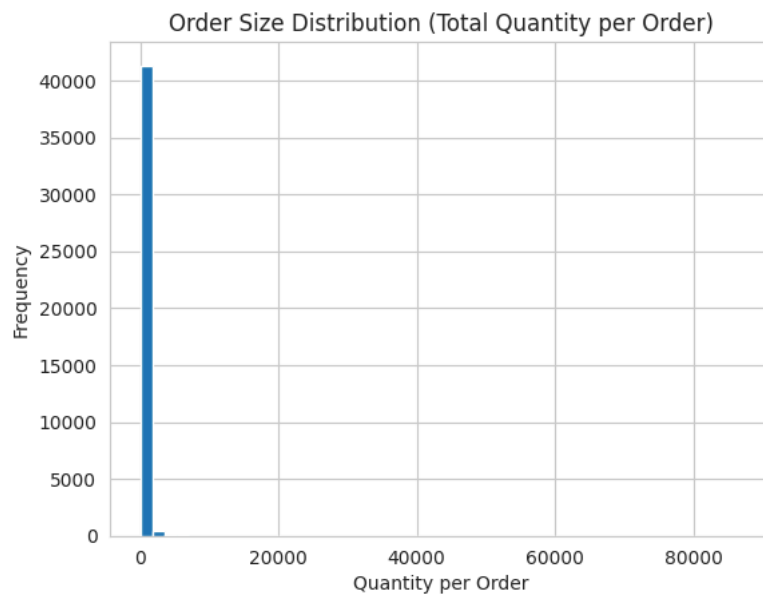
```
items_per_order = sales_df.groupby('Invoice')['Quantity'].sum()
```

```
items_per_order.describe()
```

	Quantity
count	41944.000000
mean	278.289386
std	1181.963981
min	1.000000
25%	58.000000
50%	143.000000
75%	286.000000
max	87167.000000

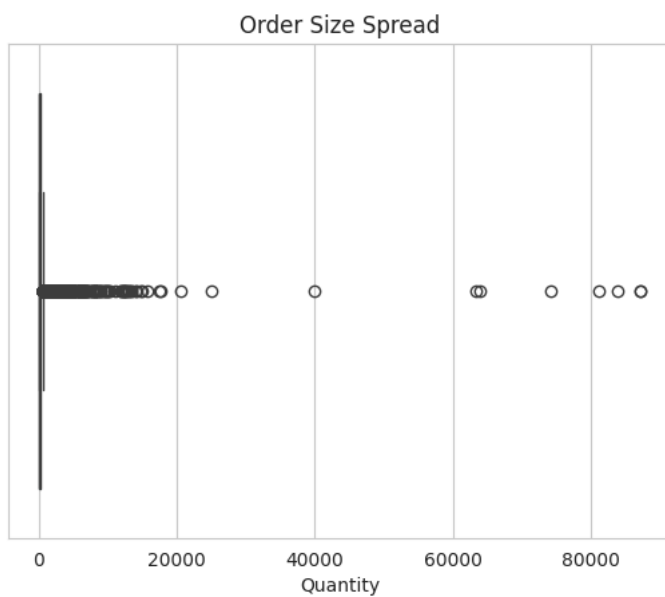
dtype: float64

```
items_per_order.plot(kind='hist', bins=50)
plt.title("Order Size Distribution (Total Quantity per Order)")
plt.xlabel("Quantity per Order")
plt.show()
```



Order sizes vary significantly, with a substantial number of bulk purchases, suggesting the presence of wholesale or high-volume buyers.

```
sns.boxplot(x=items_per_order)
plt.title("Order Size Spread")
plt.show()
```



```
sales_df.sort_values(by='Quantity', ascending=False).head(10)
```

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country	TotalPrice	year_month	revenue
1065882	581483	23843	PAPER CRAFT , LITTLE BIRDIE	80995	2011-12-09 09:15:00	2.08	16446.0	United Kingdom	168469.6	2011-12	168469.6
587080	541431	23166	MEDIUM CERAMIC TOP STORAGE JAR	74215	2011-01-18 10:01:00	1.04	12346.0	United Kingdom	77183.6	2011-01	77183.6
90857	497946	37410	BLACK AND WHITE PAISLEY FLOWER MUG	19152	2010-02-15 11:57:00	0.10	13902.0	Denmark	1915.2	2010-02	1915.2
127166	501534	21099	SET/6 STRAWBERRY PAPER CUPS	12960	2010-03-17 13:09:00	0.10	13902.0	Denmark	1296.0	2010-03	1296.0
127168	501534	21091	SET/6 WOODLAND PAPER PLATES	12960	2010-03-17 13:09:00	0.10	13902.0	Denmark	1296.0	2010-03	1296.0
127169	501534	21085	SET/6 WOODLAND PAPER CUPS	12744	2010-03-17 13:09:00	0.10	13902.0	Denmark	1274.4	2010-03	1274.4
127170	501534	21088	ASSTD DESIGN 3D	12744	2011-11-25 13:09:00	0.10	13902.0	United Kingdom	1274.4	2011-11	1274.4

Certain customers purchase coordinated product sets in bulk quantities, indicating event-based or reseller-driven demand patterns.

Sales Summary

Sales exhibit strong seasonality with peaks in Q4, particularly November. Customer activity is highly skewed, with a small number of high-frequency buyers. Product demand is concentrated among a few items, and order sizes indicate a mix of retail and bulk purchasing behavior.

REVENUE ANALYSIS

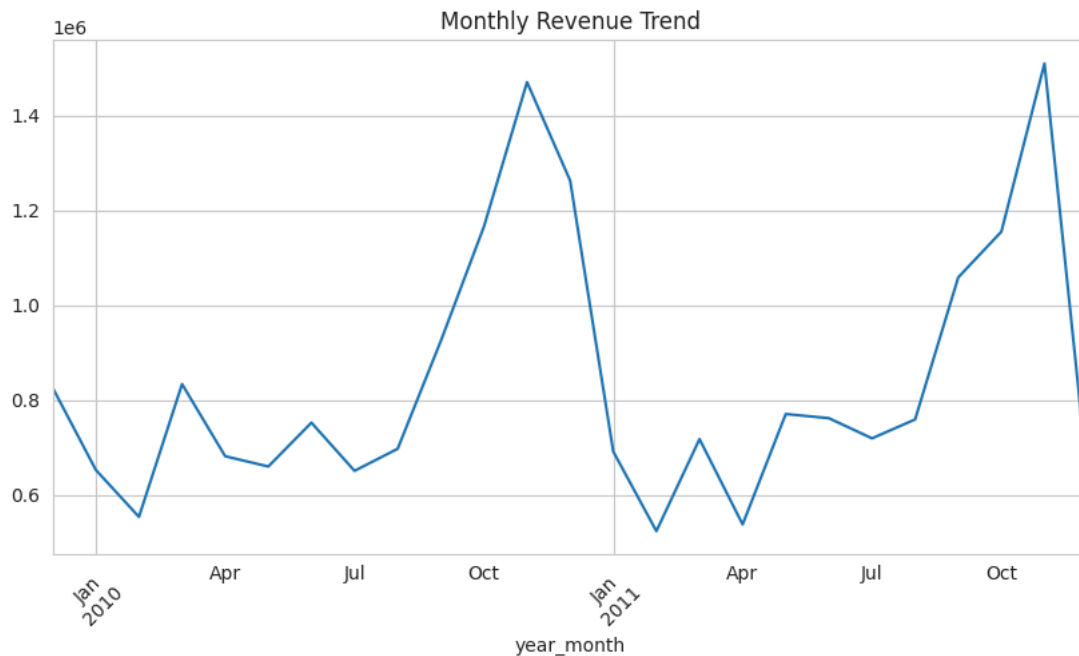
The dataset reflects a hybrid retail-wholesale model, with a small number of customers making large-volume purchases, significantly influencing overall sales distribution.

```
revenue_df = sales_df[sales_df['Price'] > 0].copy()

revenue_df['revenue'] = revenue_df['Quantity'] * revenue_df['Price']
```

Revenue Trend over Time

```
monthly_revenue = revenue_df.groupby('year_month')['revenue'].sum()
monthly_revenue.plot(figsize=(10,5))
plt.title("Monthly Revenue Trend")
plt.xticks(rotation=45)
plt.show()
```



As seen in the monthly sales trend the revenue also follows the same.

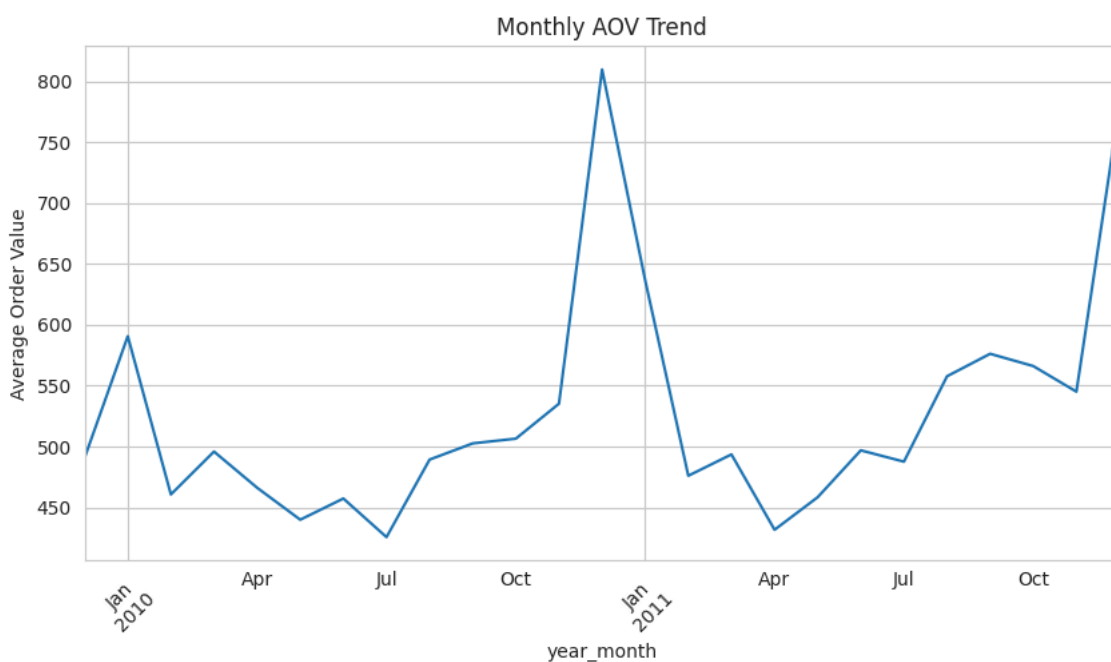
But what drives the revenue- orders or average order value?

Revenue vs Orders

```
monthly_revenue = revenue_df.groupby('year_month')['revenue'].sum()
monthly_orders = revenue_df.groupby('year_month')['Invoice'].nunique()

monthly_aov = monthly_revenue / monthly_orders
```

```
monthly_aov.plot(figsize=(10,5), title="Monthly AOV Trend")
plt.xticks(rotation=45)
plt.ylabel("Average Order Value")
plt.show()
```



Revenue growth is driven by both increasing order volume and higher average order values, indicating the influence of bulk purchasing behavior.

Top Products by Revenue

```
top_products_rev = revenue_df.groupby('Description')['revenue'].sum().sort_values(ascending=False)

top_products_rev
```

	revenue
Description	
REGENCY CAKESTAND 3 TIER	344563.250
Manual	341104.900
DOTCOM POSTAGE	322657.480
WHITE HANGING HEART T-LIGHT HOLDER	266923.550
PAPER CRAFT , LITTLE BIRDIE	168469.600
...	...
CAT W SUNGLASSES BLANK CARD	0.760
SET 12 COLOURING PENCILS DOILEY	0.650
PINK HEART CHRISTMAS DECORATION	0.380
HAPPY BIRTHDAY GINGER CAT CARD	0.380
PADS TO MATCH ALL CUSHIONS	0.017

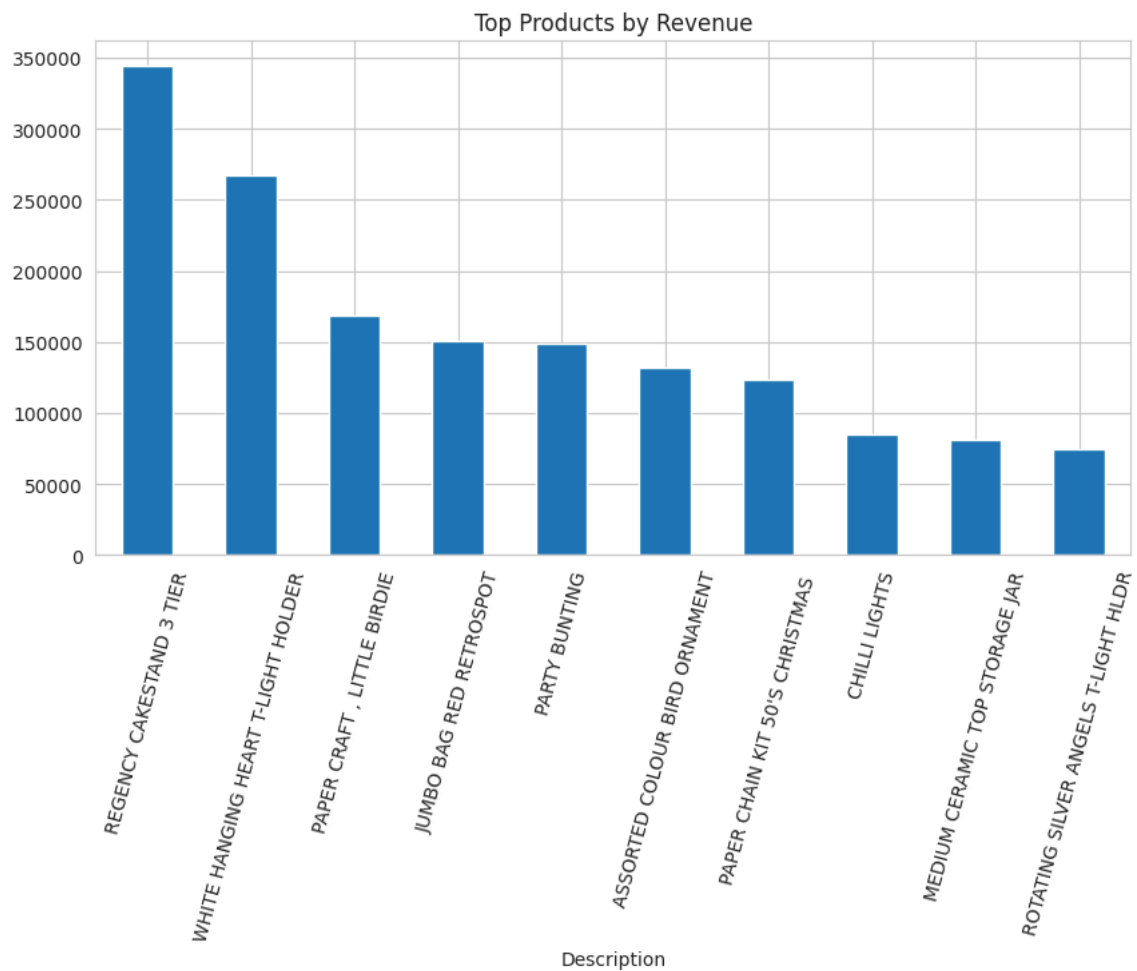
5399 rows × 1 columns

dtype: float64

```
revenue_df_clean = revenue_df[
    ~revenue_df['Description'].str.contains('postage|given away|discount|wrongly|missing|samples|charges|fee|dotcom|unsaleable')
    & (revenue_df['Description'].str.strip() != '?')
]
```

```
top_products_rev = revenue_df_clean.groupby('Description')['revenue'].sum().sort_values(ascending=False)
```

```
top_products_rev.head(10).plot(kind='bar', figsize=(10,5))
plt.title("Top Products by Revenue")
plt.xticks(rotation=75)
plt.show()
```



High-selling products are not necessarily the highest revenue generators.

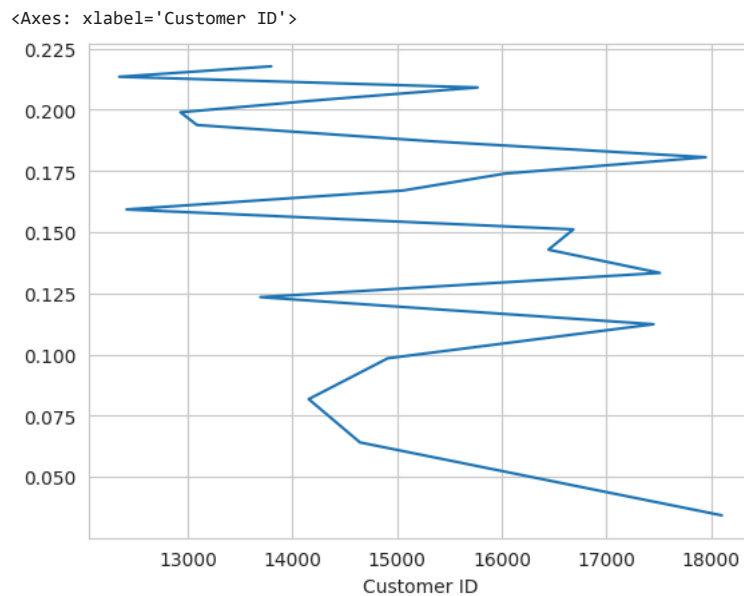
- Some products drive traffic (volume)
- Some products drive profit (revenue)

The business relies on a mix of high-volume low-price items and lower-volume higher-value items, indicating a diversified product revenue strategy.

Revenue Concentration

```
customer_revenue = revenue_df.groupby('Customer ID')['revenue'].sum().sort_values(ascending=False)
cumulative_rev = customer_revenue.cumsum() / customer_revenue.sum()
```

```
cumulative_rev.head(20).plot()
```



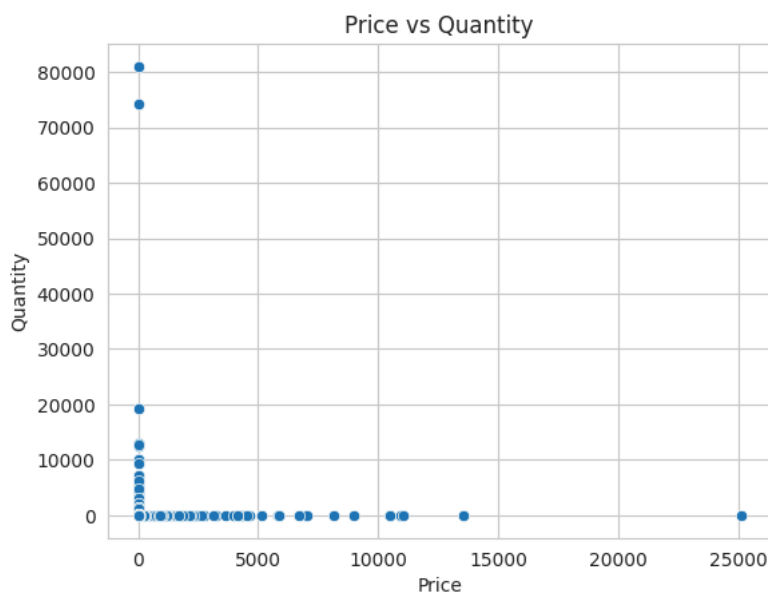
Revenue is highly concentrated among a small group of customers, indicating strong dependence on key high-value buyers.

This matters because-

- Risk: losing few customers = big loss
- Opportunity: target similar customers

✓ Price vs Quantity Relationship

```
sns.scatterplot(x='Price', y='Quantity', data=revenue_df)
plt.title("Price vs Quantity")
plt.show()
```



The business primarily follows a high-volume, low-price model, with occasional high-value transactions.

Revenue Summary

Revenue is influenced by both order volume and average order value, with bulk purchasing playing a significant role. While high-volume products drive sales, they are not always the top revenue contributors, indicating a mix of low-price and higher-value products. Additionally, revenue is highly concentrated among a small number of customers, highlighting dependency on key buyers and a hybrid retail-wholesale business model.

✓ RETURNS ANALYSIS

```
returns_df = df[df['Quantity'] < 0].copy()

returns_df['return_value'] = returns_df['Quantity'] * returns_df['Price']
```

▼ Top Returned Products

```
top_returns = returns_df.groupby('Description')['Quantity'].sum().sort_values()

top_returns.head(10)
```

	Quantity
Description	
PAPER CRAFT , LITTLE BIRDIE	-80995
MEDIUM CERAMIC TOP STORAGE JAR	-74494
?	-37641
given away	-20000
printing smudges/thrown away	-19200
ROTATING SILVER ANGELS T-LIGHT HLDR	-18750
missing	-16467
Unsaleable, destroyed.	-15644
ebay sales	-13630
check	-13329

dtype: int64

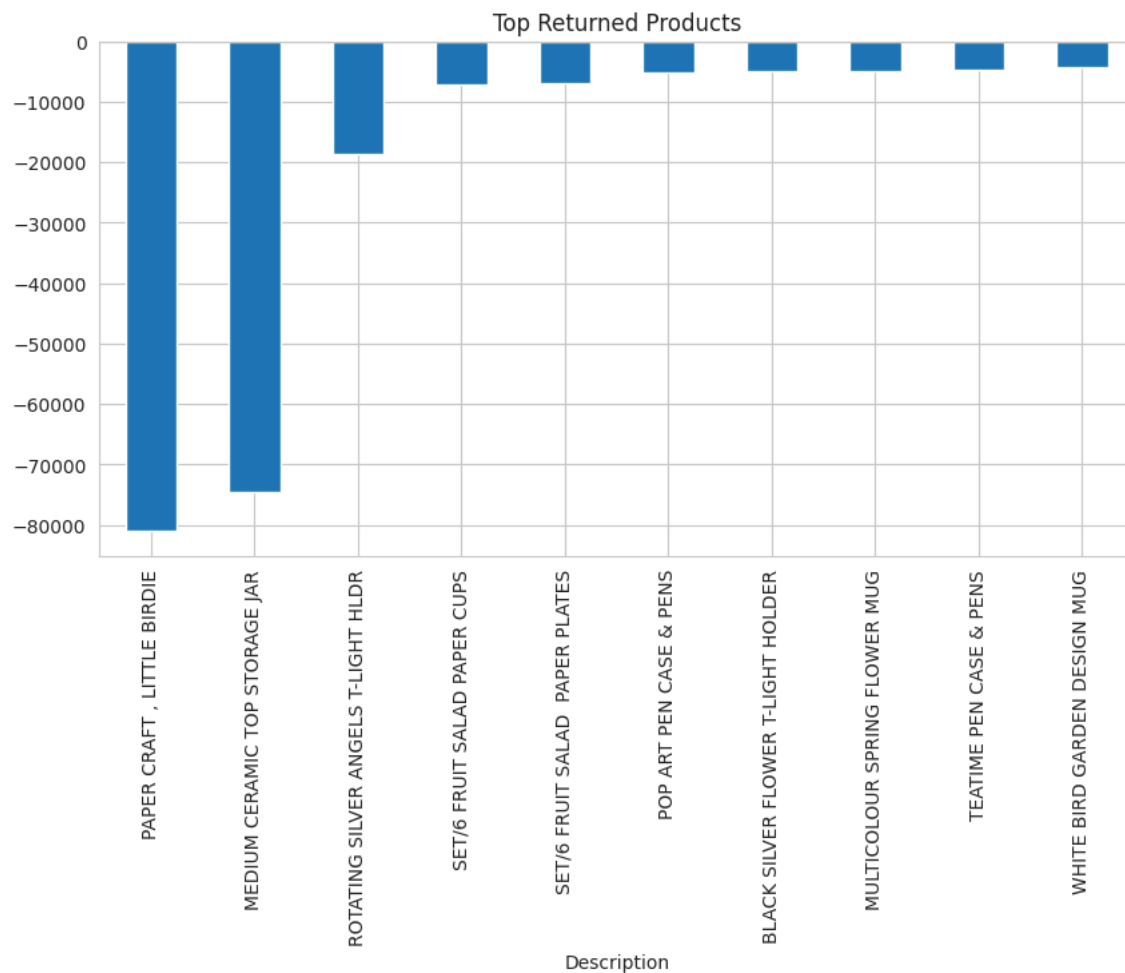
A significant portion of recorded returns consists of operational and administrative entries such as damaged, missing, or promotional items. These are being excluded to accurately analyze true customer return behavior.

```
invalid_returns = returns_df[
    returns_df['Description'].str.contains(
        'given away|discount|wrongly|missing|samples|charges|fee|dotcom|unsaleable|damaged|damages|manual|error|thrown away
        case=False,
        na=False
    )
    |
    (returns_df['Description'].str.strip() == '?')
]

valid_returns = returns_df.drop(invalid_returns.index)
```

```
top_return_products = valid_returns.groupby('Description')['Quantity'].sum().sort_values()

top_return_products.head(10).plot(kind='bar', figsize=(10,5))
plt.title("Top Returned Products")
plt.show()
```

Return Rate per Product

```
total_returns = abs(valid_returns['return_value'].sum())
total_revenue = revenue_df['revenue'].sum()

return_rate = total_returns / total_revenue
print(return_rate)
```

0.035748175225688246

Returns account for approximately 3.6% of total revenue, indicating a relatively low impact on overall business performance.

Monthly Return Trend

```
valid_returns['InvoiceDate'] = pd.to_datetime(valid_returns['InvoiceDate'])
valid_returns['year_month'] = valid_returns['InvoiceDate'].dt.to_period('M')
monthly_returns = valid_returns.groupby('year_month')['return_value'].sum().abs()

monthly_returns.plot(figsize=(10,5), title="Monthly Returns Trend")
plt.xticks(rotation=45)
plt.show()
```

